Zestaw 4

Zadanie 1. Błądzenie przypadkowe w 2D oraz 3D, RandomChoice, Position, oraz inne

Zadanie jest kontynuacją zad. 5 z zestawu nr 3, „Błądzenie przypadkowe w 1D".
Analogicznie do błądzenia przypadkowego w 1D zdefiniujmy dyskretne błądzenie kulki w dwóch wymiarach.

In[•]:=
```
getSteps[n_] := RandomChoice[{-1, 1}, n];
getWalk[n_] := Module[
  {steps, walk},
  steps = getSteps[n];
  walk = Accumulate[steps];
  walk
 ]
```

Kulka wykonuje skoki w lewo, w prawo, w dół oraz w górę z takim samym prawdopodobieństwem **0.25**.
Przykładowo skok w prawo można przedstawić jako $\{\Delta x, \Delta y\} = \{1, 0\}$, a skok w górę jako $\{\Delta x, \Delta y\} = \{0, 1\}$.
Analogicznie do funkcji `getSteps[n_]` oraz `getWalk[n_]` dla 1D zdefiniuj funkcje `getSteps2D[`nn`_]` oraz `getWalk2D[n_]` zwracające pozycję kulki po 1, 2, ... n skokach.

In[•]:=
```
getSteps2D[n_] := RandomChoice[{{-1, 0}, {1, 0}, {0, -1}, {0, 1}}, n];
getWalk2D[n_] := Module[
  {steps, walk},
  steps = getSteps2D[n];
  walk = Accumulate[steps];
  walk
 ]
```

In[•]:=
```
getWalk2D[10]
```

Out[•]=
```
{{-1, 0}, {-2, 0}, {-2, 1}, {-2, 0}, {-2, -1}, {-1, -1}, {-2, -1}, {-3, -1}, {-4, -1}, {-4, 0}}
```

Sprawdź działanie funkcji getWalk2D przedstawiając jej wynik na wykresie. Użyj w tym celu:

1) Funkcji `ListLinePlot`

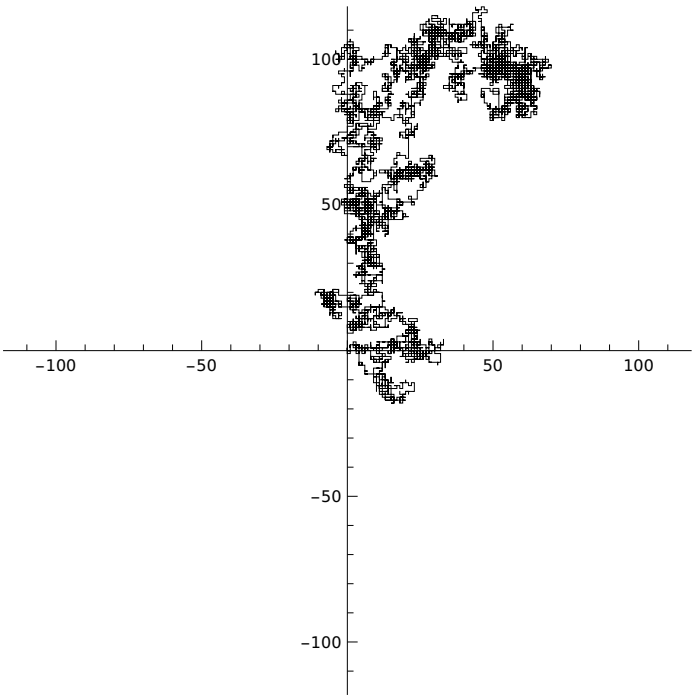
Zdefiniuj proporcje wykresu `AspectRatio → 1`, oraz określ zakres wykresu tak by obejmował kwadratowy obszar ze środkiem w punkcie **{0, 0}** i by w obszarze tym było widoczne całe błądzenie

przypadkowe.

Dla dużego n zmniejsz grubość linii korzystając z `PlotStyle`.

In[•]:=
```
walk = getWalk2D[10 * 1000];
PlotWalk2D[walk_] := Module[
   {maxValue},
   maxValue = Max[Abs[walk]];
   ListLinePlot[
    walk,
    AspectRatio → 1,
    PlotStyle → {Black, Thin},
    PlotRange → {{-maxValue, maxValue}, {-maxValue, maxValue}}
   ]
  ];
PlotWalk2D[walk]
```

Out[•]=

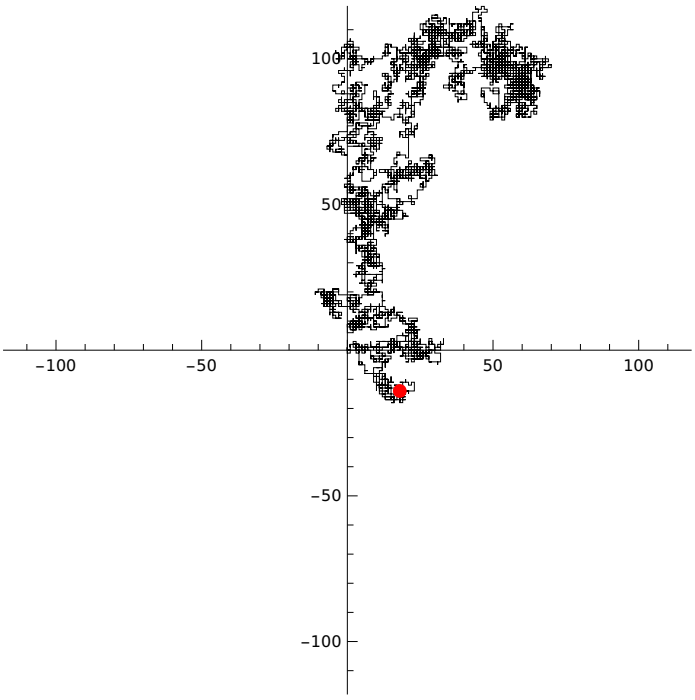


2) Korzystając z opcji Epilog, zaznacz czerwonym punktem pozycję kulki po n skokach

In[•]:=
```
PlotWalk2D[walk_] := Module[
   {maxValue},
   maxValue = Max[Abs[walk]];
   ListLinePlot[
    walk,
    AspectRatio → 1,
    PlotStyle → {Black, Thin},
    PlotRange → {{-maxValue, maxValue}, {-maxValue, maxValue}},
    Epilog → {
      Red,
      PointSize[0.02],
      Point[Last[walk]]
     }
   ]
  ];
```

```
PlotWalk2D[walk]
```

Out[•]=



3) Sprawdź czy dane błądzenie przypadkowe powraca do położenia początkowego

Sprawdź czy dane błądzenie przypadkowe powraca do położenia początkowego, a jeśli tak to po ilu krokach. Wykorzystaj funkcję `Position`.

In[•]:=
```
? Position
```

Out[•]=

Symbol

Position[*expr*, *pattern*] gives a list of the positions at which objects matching *pattern* appear in *expr*.

Position[*expr*, *pattern*, *levelspec*] finds only objects that appear on levels specified by *levelspec*.

Position[*expr*, *pattern*, *levelspec*, *n*] gives the positions of the first *n* objects found.

Position[*pattern*] represents an operator form of Position that can be applied to an expression.

In[•]:=
```
Position[walk, {0, 0}]
```

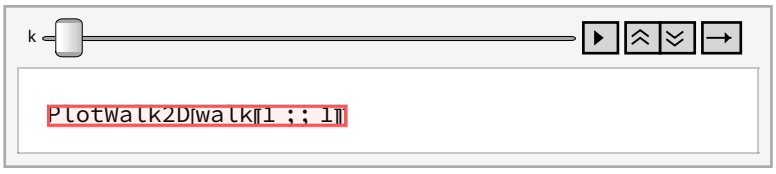
Out[•]=
```
{{2}, {4}, {6}, {8}, {10}}
```

4) Stwórz animację błądzenia przypadkowego przy pomocy funkcji `Animate`

Stwórz animację błądzenia przypadkowego przy pomocy funkcji `Animate` pokazującą jak przebiega błądzenie przypadkowe w kolejnych skokach od 1 do n = 10000.
Wykorzystaj wykres z **pkt. 1)** dla zwiększającej się liczby pokazanych punktów (skoków). Ta liczba to parametr, który będzie się zmieniał w funkcji `Animate`.

In[•]:=
```
Animate[
 PlotWalk2D[walk[[ ;; k]]],
 {k, 1, Length[walk], 1},
 AnimationRunning → False,
 DefaultDuration → 10,
 AnimationRepetitions → 1
]
```

Out[•]=

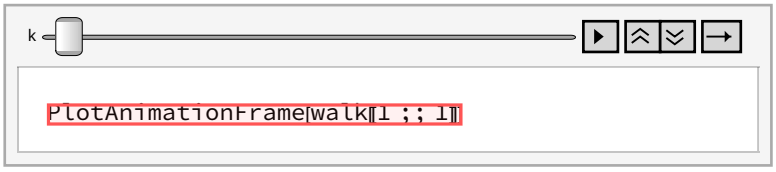


5) Na wykresie w animacji nanieś niebieski punkt w momencie, kiedy błądzenie przypadkowe powróci po raz pierwszy do punktu początkowego

Od tego momentu punkt pozostaje na wykresie. Jeśli błądzenie przypadkowe nie powraca, wtedy punkt się nie pojawia.

In[•]:=
```
PlotAnimationFrame[walk_] := Module[
    {maxValue, backAtBegining},
    maxValue = Max[Abs[walk]];
    backAtBegining = Flatten[Position[walk, {0, 0}]];
    ListLinePlot[
     walk,
     AspectRatio → 1,
     PlotStyle → {Black, Thin},
     PlotRange → {{-maxValue, maxValue}, {-maxValue, maxValue}},
     Epilog → {
       Red,
       PointSize[0.02],
       Point[Last[walk]],
       Blue,
       PointSize[Length[backAtBegining] * 0.02],
       Point[{0, 0}]
      }
    ]
   ];
Animate[
 PlotAnimationFrame[walk[[ ;; k]]],
 {k, 1, Length[walk], 1},
 AnimationRunning → False,
 DefaultDuration → 10,
 AnimationRepetitions → 1
]
```

Out[•]=



6) Stwórz analogiczną symulację z $n = 2000$ dla błądzenia przypadkowego w 3D

In[•]:=
```
movesIn3D = Flatten[Map[
    direction ↦ {direction, -direction},
    {{1, 0, 0}, {0, 0, 1}, {0, 1, 0}}
   ], 1]
```

Out[•]= {{1, 0, 0}, {-1, 0, 0}, {0, 0, 1}, {0, 0, -1}, {0, 1, 0}, {0, -1, 0}}

In[•]:=
```
getSteps3D[n_] := RandomChoice[movesIn3D, n];
getWalk3D[n_] := Module[
   {steps, walk3D},
   steps = getSteps3D[n];
   walk3D = Accumulate[steps];
   walk3D
  ];
getWalk3D[10]
```

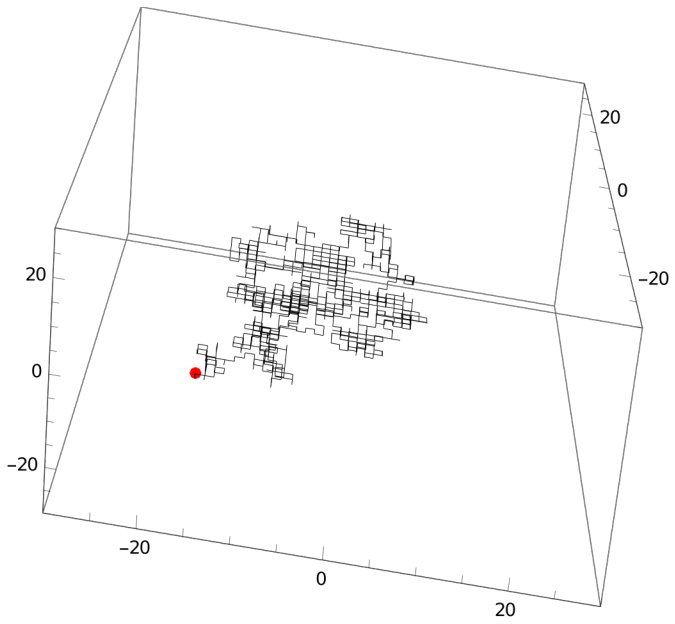
Out[•]=
```
{{0, 1, 0}, {0, 1, -1}, {0, 1, -2}, {0, 1, -3},
 {1, 1, -3}, {1, 2, -3}, {1, 2, -2}, {2, 2, -2}, {2, 2, -3}, {2, 2, -4}}
```

Użyj `ListLinePlot3D`. Do naniesienia punktów zamiast `Epilog` wykorzystaj funkcję Show.

In[•]:=
```
n = 2000;
walk3D = getWalk3D[n];
maxValue = Max[Abs[walk3D]];
backAtBegining = Flatten[Position[walk3D, {0, 0, 0}]];

Show[
 ListLinePlot3D[
  walk3D,
  PlotStyle → {Black, Thin},
  AspectRatio → 1,
  PlotRange → {{-maxValue, maxValue}, {-maxValue, maxValue}, {-maxValue, maxValue}}
 ],
 Graphics3D[{
   Red,
   PointSize[0.02],
   Point[Last[walk3D]],
   Blue,
   PointSize[Length[backAtBegining] * 0.02],
   Point[{0, 0, 0}]
  }]
]
```

Out[•]=



Zadanie 2. Pochodna jako granica, notacja, Limit, D

Pochodną funkcji $y = f(x)$ w punkcie x_0 nazywamy granicę (o ile istnieje):

$$f'(x_0) = \lim_{\Delta x \to 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$

Co symbolicznie zapisuje się m.in. jako $f'(x_0)$, $y'(x_0)$, $\frac{\mathrm{d}f}{\mathrm{d}x}\Big|_{x=x_0}$, $\frac{\mathrm{d}}{\mathrm{d}x} f(x)\Big|_{x=x_0}$, $D_x f(x_0)$.

Jeśli pochodna jest po czasie spotyka się też zapis $\dot{f}(x_0)$.

Policz pochodną dla przykładowej funkcji $f(x) = a\,x^n$ korzystając z różnych notacji w Mathematica®:

In[•]:=
```
Clear[n];
f[x_] = a x^n
```

Out[•]= $a\,x^n$

1) skorzystaj z funkcji `Limit` by zdefiniować funkcję `fD[f_, x_]` liczącą pochodną f z definicji

In[•]:=
```
? Limit
```

Out[•]=

Symbol

Limit[f, $x \to x^*$] gives the limit $\lim_{x \to x^*} f(x)$.

Limit[f, $\{x_1 \to x_1^*, \ldots, x_n \to x_n^*\}$] gives the nested limit $\lim_{x_1 \to x_1^*} \cdots \lim_{x_n \to x_n^*} f(x_1, \ldots, x_n)$.

Limit[f, $\{x_1, \ldots, x_n\} \to \{x_1^*, \ldots, x_n^*\}$] gives the multivariate limit $\lim_{\{x_1,\ldots,x_n\} \to \{x_1^*,\ldots,x_n^*\}} f(x_1, \ldots, x_n)$.

In[•]:=
```
fD[f_, x_] := Limit[
  (f[x + dx] - f[x]) / dx,
  dx → 0
 ]
fPrim = fD[f, x]
```

Out[•]= $a\,n\,x^{-1+n}$

2) skorzystaj z wbudowanej funkcji D

Zapoznaj się z jej składnią.

In[•]:= **? D**

Out[•]=

Symbol

D[f, x] gives the partial derivative $\partial f/\partial x$.

D[f, {x, n}] gives the multiple derivative $\partial^n f/\partial x^n$.

D[f, x, y, ...] gives the partial derivative $\cdots (\partial/\partial y)(\partial/\partial x) f$.

D[f, {x, n}, {y, m}, ...] gives the multiple partial derivative $\cdots (\partial^m/\partial y^m)(\partial^n/\partial x^n) f$.

D[f, {{x_1, x_2, ...}}] for a scalar f gives the vector derivative $(\partial f/\partial x_1, \partial f/\partial x_2, \ldots)$.

D[f, {$array$}] gives an array derivative.

In[•]:= **D[f[x], x]**

Out[•]= a n x^{-1+n}

3) policz pochodną drugiego, trzeciego, k-tego rzędu korzystając z D

$f''(x)$

In[•]:= **D[f[x], {x, 2}]**

Out[•]= a (−1 + n) n x^{-2+n}

$f'''(x)$

In[•]:= **D[f[x], {x, 3}]**

Out[•]= a (−2 + n) (−1 + n) n x^{-3+n}

$f^{(k)}(x)$

In[•]:= **D[f[x], {x, k}]**

Out[•]= a x^{-k+n} FactorialPower[n, k]

4) skorzystaj z zapisu f'[x], f''[x], f'''[x]

In[•]:= **f'[x]**

Out[•]= a n x^{-1+n}

In[•]:= **f''[x]**

Out[•]= a (−1 + n) n x^{-2+n}

In[•]:= **f'''[x]**

Out[•]= a (−2 + n) (−1 + n) n x^{-3+n}

5) skorzystaj z Derivative[rząd_pochodnej_][funkcja_]

```
In[•]:= Derivative[2][f]
        Derivative[3][f]
        Derivative[k][f]
Out[•]= a (-1 + n) n #1^(-2+n) &
Out[•]= a (-2 + n) (-1 + n) n #1^(-3+n) &
Out[•]= a FactorialPower[n, k] #1^(-k+n) &
```

Ciekawostka:
W poleceniu `Derivative` *ujemny rząd pochodnej będzie reprezentował całkę nieoznaczoną odpowiedniego rzędu z zadanej funkcji.*

```
In[•]:= Derivative[-1][f]
Out[•]= (a #1^(1+n))/(1 + n) &
```

Zadanie 3. Pochodna i jej własności, D

Teraz skorzystaj z wbudowanej funkcji D by policzyć następujące pochodne dla funkcji fA(x)oraz fB(x):

1) pochodna sumy funkcji (fA + fB)′ $\left(x\right)$

```
In[•]:= D[fA[x] + fB[x], x]
Out[•]= fA'[x] + fB'[x]
```

2) pochodna iloczynu funkcji $\left(\text{fA fB}\right)'\left(x\right)$

```
In[•]:= D[fA[x] * fB[x], x]
Out[•]= fB[x] fA'[x] + fA[x] fB'[x]
```

3) pochodna odwrotności funkcji $\left(\frac{1}{\text{fA}}\right)'\left(x\right)$

```
In[•]:= D[1 / fA[x], x]
Out[•]= -(fA'[x]/fA[x]^2)
```

4) pochodna ilorazu funkcji $\left(\frac{fA}{fB}\right)'(x)$

```
In[•]:= FullSimplify[D[fA[x] / fB[x], x]]
```

$$\text{Out[•]=}\ \frac{fB[x]\,fA'[x] - fA[x]\,fB'[x]}{fB[x]^2}$$

5) pochodna złożenia funkcji (fA(fB))$'(x)$

```
In[•]:= D[fA[fB[x]], x]
```

```
Out[•]= fA'[fB[x]] fB'[x]
```

Zadanie 4. Pochodne w punkcie, D

Obliczyć pochodne do 6-tego rzędu dla funkcji.
Podać wartość dokładną, a następnie przybliżoną z użyciem N.

1) $f(x) = x^x$ w punkcie $x = 1$

```
In[•]:= Derivative[6][Function[x, x^x]][1]
```

```
Out[•]= 54
```

2) $f(x) = \ln^4\left(\frac{x}{(x+e)^2}\right)$ w punkcie $x = e$

```
In[•]:= result = FullSimplify[
          Derivative[6][Function[x, Log[x / (x + E)^2] ^ 4]][E]
        ]
```

$$\text{Out[•]=}\ \frac{15\,(1 + \text{Log}[4])\,(167 + \text{Log}[4]\,(223 + 62\,\text{Log}[4]))}{2\,e^6}$$

```
In[•]:= N[result]
```

```
Out[•]= 26.409
```

```
In[•]:= FullSimplify[res]
```

```
Out[•]= res
```

Zadanie 5. Pochodna jako styczna w punkcie, D, Plot

Znaleźć równanie stycznej do wykresu funkcji $f(x) = \sin x$ w punkcie $x = 2$.

In[•]:=
```
f[x_] = Sin[x];
x0 = 2;
sA = f'[x0];
sB = f[x0] - sA x0;
s[x_] = sA x + sB
```

Out[•]=
```
-2 Cos[2] + x Cos[2] + Sin[2]
```

Przedstawić na wykresie funkcję f, wyznaczoną styczną oraz zaznaczyć punkt styczności. Proszę wykonać wykres w zakresie $-4 \le x \le 8$.

In[•]:=
```
Plot[
 {f[x], s[x]},
 {x, -4, 8},
 Epilog → {
   Red,
   PointSize[0.02],
   Point[{x0, f[x0]}]
  }
]
```

Out[•]=

